

maix.app

maix.app module

You can use `maix.app` to access this module with MaixPy
This module is generated from `MaixPy` and `MaixCDK`

1. Module

No module

2. Enum

3. Variable

4. Function

4.1. app_id

```
1 | def app_id() -> str
```

Get current APP ID.

item	description
return	APP ID.

C++ definition code:

```
1 | string app_id()
```

4.2. set_app_id

```
1 | def set_app_id(app_id: str) -> str
```

Set current APP ID.

item	description
param	app_id: APP ID.

C++ definition code:

```
1 | string set_app_id(const string  
    &app_id)
```

4.3. get_apps_info_path

```
1 | def get_apps_info_path() -> str
```

Get APP info file path.

C++ definition code:

```
1 | string get_apps_info_path()
```

4.4. get_apps_info

```

1 | def get_apps_info(ignore_launcher:
    | bool = False, ignore_app_store: bool
    | = False) -> list[APP_Info]

```

Get APP info list.

item	description
param	ignore_launcher : if true, ignore launcher APP. default false. ignore_app_store : if true, ignore app store APP. default false.
return	APP info list. APP_Info object list.

C++ definition code:

```

1 | vector<app::APP_Info>
    | &get_apps_info(bool
    | ignore_launcher = false, bool
    | ignore_app_store = false)

```

4.5. get_app_info

```

1 | def get_app_info(app_id: str) ->
    | APP_Info

```

Get app info by app id.

item	description
return	app.APP_Info type.

C++ definition code:

```
1 | app::APP_Info get_app_info(const  
std::string &app_id)
```

4.6. get_app_data_path

```
1 | def get_app_data_path() -> str
```

Get APP info, APP can store private data in this directory.

item	description
return	APP data path "./data", just return the data folder in current path because APP executed in app install path or project path. So, you must execute your program in you project path to use the project/data folder when you debug your APP.

C++ defination code:

```
1 | string get_app_data_path()
```

4.7. get_app_path

```
1 | def get_app_path(app_id: str = '') -  
> str
```

Get APP path.

item	description
param	app_id : APP ID, if empty, return current APP path, else return the APP path by app_id.
return	APP path, just return the current path because APP executed in app install path or project path. So, you must execute your program in you project path to use the project/data folder when you debug your APP.

C++ defination code:

```
1 | string get_app_path(const string
    &app_id = "")
```

4.8. get_tmp_path

```
1 | def get_tmp_path() -> str
```

Get global temporary data path, APPs can use this path as temporary data directory.

item	description
return	temporary data path.

C++ defination code:

```
1 | string get_tmp_path()
```

4.9. get_share_path

```
1 | def get_share_path() -> str
```

Get data path of share, shared data like picture and video will put in this directory

item	description
return	share data path.

C++ definition code:

```
1 | string get_share_path()
```

4.10. get_picture_path

```
1 | def get_picture_path() -> str
```

Get picture path of share, shared picture will put in this directory

item	description
return	share picture path.

C++ definition code:

```
1 | string get_picture_path()
```

4.11. get_video_path

```
1 | def get_video_path() -> str
```

Get video path of share, shared video will put in this directory

item	description
return	share video path.

C++ definition code:

```
1 | string get_video_path()
```

4.12. get_font_path

```
1 | def get_font_path() -> str
```

Get font path of share, shared font will put in this directory

item	description
return	share font path.

C++ definition code:

```
1 | string get_font_path()
```

4.13. get_icon_path

```
1 | def get_icon_path() -> str
```

Get icon path of share, shared icon will put in this directory

item	description
return	share icon path.

C++ definition code:

```
1 | string get_icon_path()
```

4.14. get_sys_config_kv

```
1 | def get_sys_config_kv(item: str,
    key: str, value: str = '',
    from_cache: bool = True) -> str
```

Get system config item value.

item	description
param	item: name of setting item, e.g. wifi, language. more see settings APP. key: config key, e.g. for wifi, key can be ssid, for language, key can be locale. value: default value, if not found, return this value. from_cache: if true, read from cache, if false, read from file.
return	config value, always string type, if not found, return empty string.

C++ definition code:

```
1 | string get_sys_config_kv(const
    string &item, const string &key,
```



```
const string &value = "", bool  
from_cache = true)
```

4.15. get_app_config_kv

```
1 | def get_app_config_kv(item: str,  
    key: str, value: str = '',  
    from_cache: bool = True) -> str
```

Get APP config item value.

item	description
param	item: name of setting item, e.g. user_info key: config key, e.g. for user_info, key can be name, age etc. value: default value, if not found, return this value. from_cache: if true, read from cache, if false, read from file.
return	config value, always string type, if not found, return empty string.

C++ definition code:

```
1 | string get_app_config_kv(const  
    string &item, const string &key,  
    const string &value = "", bool  
    from_cache = true)
```

4.16. set_app_config_kv

```
1 | def set_app_config_kv(item: str,
    key: str, value: str, write_file:
    bool = True) -> maix.err.Err
```

Set APP config item value.

item	description
param	item: name of setting item, e.g. user_info key: config key, e.g. for user_info, key can be name, age etc. value: config value, always string type. write_file: if true, write to file, if false, just write to cache.
return	err::Err

C++ definition code:

```
1 | err::Err set_app_config_kv(const
    string &item, const string &key,
    const string &value, bool
    write_file = true)
```

4.17. get_app_config_path

```
1 | def get_app_config_path() -> str
```

Get APP config path, ini format, so you can use your own ini parser to parse it like `configparser` in Python.\nAll APP config info is recommended to store in this file.

item	description
return	APP config path(ini format).

C++ definition code:

```
1 | string get_app_config_path()
```

4.18. set_exit_msg

```
1 | def set_exit_msg(code: maix.err.Err,
    msg: str) -> maix.err.Err
```

Set APP exit code and exit message.\nIf code != 0, the launcher will show a dialog to user, and display the msg.

item	description
param	code : exit code, 0 means success, other means error, if code is 0, do nothing. msg : exit message, if code is 0, msg is not used.
return	exit code, the same as arg @code.

C++ definition code:

```
1 | err::Err set_exit_msg(err::Err
    code, const string &msg)
```

4.19. get_exit_msg

```
1 | def get_exit_msg(cache: bool =
    False) -> tuple[str, maix.err.Err,
    str]
```

Get APP exit code and exit message.

item	description
param	cache: if true, read from cache, if false, read from file. default false.
return	exit return app_id, exit code and exit message.

C++ defination code:

```
1 | tuple<string, err::Err, string>
    get_exit_msg(bool cache = false)
```

4.20. have_exit_msg

```
1 | def have_exit_msg(cache: bool =
    False) -> bool
```

Check if have exit msg

item	description
param	cache: if true, just check from cache, if false, check from file. default false.
return	true if have exit msg, false if not.

C++ defination code:

```
1 | bool have_exit_msg(bool cache =  
    false)
```

4.21. switch_app

```
1 | def switch_app(app_id: str, idx: int  
    = -1, start_param: str = '') -> None
```

Exit this APP and start another APP(by launcher).\nCall this API will call set_exit_flag(true), you should check app::need_exit() in your code.\nAnd exit this APP if app::need_exit() return true.

item	description
param	app_id: APP ID which will be started. app_id and idx must have one is valid. idx: APP index. app_id and idx must have one is valid. start_param: string type, will send to app, app can get this param by <code>app.get_start_param()</code>
attention	If app id or idx the same as current app, do nothing.

C++ definition code:

```
1 | void switch_app(const string  
    &app_id, int idx = -1, const  
    std::string &start_param = "")
```

4.22. get_start_param

```
1 | def get_start_param() -> str
```

Get start param set by caller

item	description
return	param, string type

C++ definition code:

```
1 | const std::string  
   get_start_param()
```

4.23. need_exit

```
1 | def need_exit() -> bool
```

Shoule this APP exit?

item	description
return	true if this APP should exit, false if not.
attention	This API is a function, not a variable.

C++ definition code:

```
1 | bool need_exit()
```

4.24. running

```
1 | def running() -> bool
```

App should running? The same as !app::need_exit()
(not app::need_exit() in MaixPy).

item	description
return	true if this APP should running, false if not.
attention	This API is a function, not a variable.
C++ defination code:	
<pre>1 bool running()</pre>	

4.25. set_exit_flag

```
1 | def set_exit_flag(exit: bool) ->  
    None
```

Set exit flag. You can get exit flag by
app.need_exit().

item	description
param	exit: true if this APP should exit, false if not.
C++ defination code:	
<pre>1 void set_exit_flag(bool exit)</pre>	

5. Class

5.1. Version

APP version

C++ definition code:

```
1 | class Version
```

5.1.1. __str__

```
1 | def __str__(self) -> str
```

Convert to string, e.g. 1.0.0

item	description
type	func
static	False

C++ definition code:

```
1 | std::string __str__()
```

5.1.2. from_str

```
1 | def from_str(version_str: str) ->
    Version
```

Convert from string, e.g. "1.0.0"

item	description
type	func
static	True

C++ definition code:

```
1 | static app::Version
   | from_str(const string
   | &version_str)
```

5.2. APP_Info

APP info

C++ definition code:

```
1 | class APP_Info
```

5.2.1. id

APP id

item	description
type	var
static	False
readonly	False

C++ definition code:

```
1 | string id
```

5.2.2. name

APP name

item	description
type	var
static	False
readonly	False

C++ defination code:

```
1 | string name
```

5.2.3. icon

APP icon

item	description
type	var
static	False
readonly	False

C++ defination code:

```
1 | string icon
```

5.2.4. version

APP version

item	description
type	var
static	False
readonly	False

C++ defination code:

```
1 | Version version
```

5.2.5. exec

APP exec

item	description
type	var
static	False
readonly	False

C++ defination code:

```
1 | string exec
```

5.2.6. author

APP author

item	description
type	var
static	False

item	description
readonly	False

C++ defination code:

```
1 | string author
```

5.2.7. desc

APP desc

item	description
type	var
static	False
readonly	False

C++ defination code:

```
1 | string desc
```

5.2.8. names

APP names

item	description
type	var
static	False
readonly	False

C++ defination code:

```
1 | map<string, string> names
```

5.2.9. desc

APP desc

item	description
type	var
static	False
readonly	False

C++ definition code:

```
1 | map<string, string> desc
```